



FINAL YEAR PROJECT
EPFL PROGRAMMING METHODS LABORATORY (LAMP)

Specialization via Specialized Traits in Scala 3

Hamish STARLING

supervised by
Dr. Solal PIRELLI
Pr. Martin ODERSKY

Spring 2026

Foreword

This project was realised during an exchange at EPFL from Imperial College London, in the framework of the Swiss-European Mobility Programme. The author is particularly grateful to the project supervisors (Dr. Pirelli and Pr. Odersky), the LAMP lab, the École Polytechnique Fédérale de Lausanne and the Swiss Confederation for their generous support in making the project a reality. Special thanks are also dedicated to Hamza Remmal and Dr Oliver Bračevac for their continued support to the project, as well as to Dr Jamie Willis at Imperial for inspiring a love of the Scala programming language.

Contents

1	Introduction	4
1.1	Specialization for Polymorphism	4
1.2	Scala 2 Specialization	6
1.3	Inline Traits [1]	7
2	Specialized Traits & Methods	9
2.1	Manual Specialization via Inline Traits	9
2.2	Specialized Traits	10
2.3	Benefits of Specialized Traits	11
3	Behaviour, Rules & Motivation	13
3.1	Creating Specialized Trait Instances	13
3.1.1	Anonymous Class Instances & Implementation Classes . .	13
3.1.2	Direct Inheritance	14
3.2	Reference Types, Specializing Supertype & Variance	14
3.3	Families of Specialized Traits & Partial Specialization	17
3.4	Transportation of Specialized through Generic Code	18
4	Modifications to Inline Traits	19
4.1	Replacement of Inline Trait Symbols	19
4.2	Additional Language Features in Inline Traits	20
4.2.1	Super Calls	20
4.2.2	Opaque Types	21
4.2.3	Self Types	22
4.3	Inlining Before Pickling & into Inline Traits	22
4.3.1	Inlining Before Pickling	22
4.3.2	Inlining into Inline Traits	23
4.3.3	Desire for Specialized Interface Traits to be Inline	24
4.4	Inline Trait Cycle Checking	26
4.5	Sharpening of Semantics	27
4.6	Bug Fixes	28
4.7	Remaining Future Work for Inline Traits	28
5	Implementation	30
5.1	<i>DesugarSpecializedTraits</i>	30
5.1.1	Enforcement of Specialized Trait Rules	30
5.1.2	Collection of Referenced Specializations	31
5.1.3	Implementation Class and Interface Trait Generation . . .	33
5.1.4	Inlining of Specialized Methods	34
5.2	<i>Erase</i>	35
5.2.1	Addition of Parents to Manually-Extending Classes	36
5.2.2	Bridge Methods	37
5.3	<i>PruneSpecializedMethods</i>	37

6	Evaluation	38
6.1	Performance	38
6.1.1	Methodology	38
6.1.2	Results	39
6.2	Generated Code	40
6.3	Ergonomics	42
7	Future Work	44
7.1	Suggested Fixes & Checks	44
7.2	Desirable Future Features	45
A	Benchmark Code	47
A.1	VectorDotProduct	47
A.2	MatrixMultiplication	49
A.3	PairHashing	51

1 Introduction

1.1 Specialization for Polymorphism

Like most programming languages, Scala supports polymorphic methods and classes which can be defined in terms of generic type arguments. For example:

```
1 trait List[T]
2 def id[T](x: T): T
```

Listing 1: Polymorphic traits and methods

For compiler writers, these must be handled carefully: values of different types have different representations, so a single machine code version of a function will typically not be compatible with multiple different data types. This leads to two principal approaches to implementing polymorphic methods:

- **Monomorphisation**, as used in C++ templates. During compilation, create a copy of the generic method or class for every type argument combination used in the program, and replace all generic functions/classes with their specialized copies. **Pros:** Better performance due to specialization. **Cons:** Larger code size, problematic for separate compilation.
- **Erasure**, as used in Java and Scala. Values of all data types are converted to a single homogenous representation (typically object pointers), which allows a single version of a function to operate on them, and a single instance of each polymorphic data type to exist.¹ **Pros:** Less code duplication, works for separate compilation. **Cons:** Overhead in conversion to homogenous representation; type-specific optimisations may be missed.

Scala adopts an erasure model in order to support separate compilation, and for compatibility with the Java Virtual Machine (JVM). However, this can impose considerable performance hazards. Consider the program in Listing 2:

```
1 class Vector[T: Numeric](elems: Array[T]):
2   private val num = summon[Numeric[T]]
3
4   def length = elems.length
5
6   def apply(i: Int): T = elems(i)
7
8   def scalarProduct(other: Vector[T]): T =
9     require(this.length == other.length)
10    var result = num.fromInt(0)
11    for i ← 0 until length do
12      result = num.plus(result, num.times(this(i), other(i)))
13    result
```

Listing 2: A generic vector [6]

¹In Java/Scala a generic type like `List[T]` erases to `List`, equivalent to `List[Object]`

Due to erasure, every value of type `T` in the source code gains implementation type `Object`. Thus, the elements of the vector must be represented as `Objects` at runtime, whether they are primitives like `int` or more elaborate reference types (e.g. `BigInt`). This slows the code down in the following ways [6]:

- To enable compatibility with primitive data types like `int` which are not `Objects`, we must *box* these (wrap them inside an `Object` such as `Integer`). To extract the `int` later requires *unboxing*. This `Object` creation and destruction has overhead, and applies not only to internal values such as `result`, but also in the API of the generic vector.
- The array `elems` is implemented as a generic array of references which has to be accessed via *reflection* (runtime inspection of types), which is much slower than accessing a primitive array such as `int[]`.
- We rely on function calls to methods of the `Numeric` typeclass instance for simple operations such as addition. In the case that a primitive data type is used in place of `T` we would like to be able to inline primitive operations instead of these method calls.

In contrast, if we manually specialize the vector as in Listing 3, these issues disappear. The `Array[Int]` will erase to `int[]` (an existing optimisation in the Scala 3 compiler); the typeclass instance is no longer needed, and there is no boxing as all references to `Int` erase directly to `int`.²

```

1 class VectorInt(elems: Array[Int]):
2   def length = elems.length
3
4   def apply(i: Int): Int = elems(i)
5
6   def scalarProduct(other: VectorInt): Int =
7     require(this.length == other.length)
8     var result = 0
9     for i ← 0 until length do
10       result = result + this(i) * other(i)
11     result

```

Listing 3: A specialized vector [6]

This translates to substantial performance gains: in a simple one-to-one benchmark on vectors of length 100,000,000, the specialized version is over 2.5× faster.³ With smaller vectors fitting in the CPU cache the difference can be as much as 30×.

For this reason there has long been interest in the performance gains that could be achieved by targeted specialization through monomorphising certain types of

²In contrast to Java, Scala does not have separate notions of boxed and unboxed primitive types in source code. The only available type is `Int`, which is boxed automatically if necessary by the compiler.

³See section 6 for in-depth benchmarking information.

methods in Scala. The idea is to allow the programmer to opt-in to automatic generation of specialized versions of their classes when the performance benefit would outweigh the cost of additional code generation. This needs to be done carefully to maintain compatibility with the erasure model in the rest of Scala.

This has manifested in two previous attempts at the specialization problem:

- Scala 2 specialization [3]
- Inline traits [1], which are the basis on which this work is built.

Several other tools have also been produced to work alongside the Scala compiler and add specialization, including recently TASTyTruffle [2] which performs Just-In-Time (JIT) specialization via a Truffle interpreter.

1.2 Scala 2 Specialization

Scala 2 supported class specialization via the `@specialized` annotation which created specialized versions of classes and methods, that were monomorphised to a number of primitive types.

```

1 class A[@specialized T] {
2   val x: T = ???
3 }
4
5 // Specializes to:
6 class A$mcI$sp extends A[Int] {
7   protected[this] val x$mcI$sp: Int = scala.Predef.???()
8   def x$mcI$sp(): Int = A$mcI$sp.this.x$mcI$sp
9   override def x(): Int = A$mcI$sp.this.x$mcI$sp()
10 }

```

Listing 4: Specialization in Scala 2 [1]

However this approach had a number of issues:

- **Diamond Problem** [4]: Specialization is applied to classes, and so suffers from the diamond problem: Suppose `B extends A`. If `A` specializes to `A_int` and `B` to `B_int`, ideally `B_int` extends both `B` and `A_int`. This is not possible as multiple class inheritance is not supported.
- **Pre-Specialization** [4]: Specializations are generated when the class is defined and not at the point of use. This has the following consequences:
 - **Code Explosion**: all possible specializations have to be generated. The number of generated classes grows exponentially with multiple specialized parameters, such as for tuples. This means that tuples larger than pairs are not specialized in Scala 2.
 - **Type Guessing**: The author of a specialized class may restrict the generated specializations to certain argument types to reduce code

explosion, but in that case if a different type is supplied by a library user there will be no specialization, so the library author must correctly guess in advance which type arguments will later be used.

- **Type Limitation:** Specialization is limited by the compiler to a subset of primitive types to reduce code explosion. Primitive types offer the most potential gain for specialization as they allow us to avoid boxing, but specialization to e.g. `String` would be worth having to avoid runtime downcasting from `Object` to `String`⁴, and to reduce megamorphic dispatch.

- **Field Duplication** [4]: Instances of the generated specialized classes must reserve space for both the generic and specialized field sets (`val x` and `val x$mcI$sp` above), which makes them larger than necessary.

1.3 Inline Traits [1]

Inline traits is a new approach for implementing specialization in Scala 3, seeking to resolve these issues, and working in a very similar way to inline methods.

The body of an inline trait is *copied* by the compiler into the body of any extending classes, and any type or term parameters are replaced by the provided arguments. This has the effect of producing specialized members in the child class according to the types provided to the type parameters of the inline trait:

```
1 inline trait A[T](val x: T):  
2   def foo: T = x  
3 class B extends A[Int](1)  
4  
5 // inlines to:  
6 class B extends A[Int](1):  
7   override val x: Int = 1  
8   override def foo: Int = this.x
```

Listing 5: Inline traits [1]

This resolves the above problems:

- **Diamond Problem:** Specialization happens through traits. These freely support multiple inheritance via mixin composition, avoiding this issue.
- **Pre-Specialization:** Specializations are only generated when another class inlines the inline trait, not pre-emptively. This resolves the issues of **Code Explosion** and **Type Guessing**.
- **Type Limitation:** The usual inlining mechanism is used, and this supports specialization to any data type.

⁴Downcasts on the JVM are not free as they are typechecked at runtime so an exception can be thrown if the cast is not viable, however the JIT compiler often optimises them out.

- **Field Duplication:** After inlining, inline traits are pruned so that they exist at runtime only as pure interfaces. This means that the child classes do not inherit unspecialized fields from their parent inline traits, and therefore do not have a duplicate field set.

However, inline traits suffer from one shortcoming, as discussed in section 2.3 of [1], namely that interfaces are not specialized:

```

1 inline trait Foo[T](x: T):
2   def foo: T = x
3
4 class Bar extends Foo[Int](42)
5
6 def f(b: Foo[Int]) = 37 + b.foo
7
8 @main def main =
9   val x = Bar()
10  f(x)

```

Listing 6: Inline traits limitation

In Listing 6, the call to `b.foo` will refer to the version of `foo` typed `foo: T` (which becomes `foo: Object` after erasure), because we accessed `foo` on an object of declared type `Foo` (even though `b`'s actual runtime type is `Bar`). This will in turn call a bridge method, which means the `foo: Int` method will be called, but unnecessary boxing and unboxing will be added:

```

1 inline trait Foo
2   def foo#1(): Object
3
4 class Bar extends Foo[Int](42):
5   override def foo#2(): Int = 42
6   override def foo#3(): Object = Int.box(this.foo())
7
8 def f(b: Foo): Int = 37 + Int.unbox(b.foo#1()) // virtual call foo#1 resolves to bridge↔
      method foo#3, which in turn calls actual method foo#2, with boxing.

```

Listing 7: Inline traits limitation compilation result

This may seem harmless but in practice it is a relatively big restriction since it means that a generic datatype such as `inline trait Map[K, V]` can never be used in interfaces without first subclassing it to the desired type arguments and instead referencing that subclass.

This limitation thus prevents inline traits from being a perfect replacement for Scala 2 specialization. This project seeks to overcome the limitation, and thus to bring specialization closer to being ready for integration into Scala 3.

2 Specialized Traits & Methods

2.1 Manual Specialization via Inline Traits

We recall the shortcoming of inline traits from Listing 6. In order to overcome this it is imperative to modify the interface of the `f` method so that it refers to a specialized type. A simple solution (solution 1 in [1] section 2.3) would be to expect the user to instead write the following:

```
1 inline trait Foo[T](x: T):  
2   def foo: T = x  
3  
4 class FooInt(x: Int) extends Foo[Int](x)  
5  
6 def f(b: FooInt) = 37 + b.foo
```

Listing 8: Manual interface specialization

However this comes with a few disadvantages:

- **Code Bloat:** Requires the user to pollute their code with these specialization classes, of which there could be a large number if multiple specialized type parameters are in use.
- **Lack of Standardisation:** A library author may create a specialization `IntMatrix`. A user of the library may not know this is available and may create `MatrixInt`. This creates unnecessary code duplication, and these two specialized classes would not be interchangeable.
- **User Responsibility:** Assuming we wish to avoid exponential explosion of specialization classes due to pre-specialization, it must be the user of a library who is responsible for creating specializations to the types in their program. They may not have the expertise to know whether this is a good idea, or may not know that it is possible.
- **Generic Specialized Methods:** Since the specialized classes must be directly referenced, we can't write methods that are both polymorphic and specialized:

```
1 inline trait Vec[T]  
2 class VecInt extends Vec[Int]  
3  
4 def times[T](x: Vec[T], y: Vec[T])  
5 // even making this method inline does not allow for specialization:  
6 inline def times[T](x: Vec[T], y: Vec[T])
```

Listing 9: Generic specialized methods

The compiler cannot relate `Vec[Int]` and `VecInt`, so we can't perform specialization automatically. The inlined body of the `times` method will be in terms of `Vec[Int]`, which erases to the unspecialized `Vec`.

2.2 Specialized Traits

To support interface specialization while avoiding these issues, we introduce specialized traits [6], specified via the `Specialized` context bound:

```
1 inline trait Foo[T: Specialized](x: T):
2   def foo: T = x
```

Listing 10: A specialized trait

The presence of a `Specialized` bound on a type parameter (only permitted in inline methods and inline traits) tells the compiler to generate additional versions of the trait specialized to the concrete type supplied. These specializations are generated on-demand based on type arguments supplied in the program.

```
1 inline trait Foo$sp$Int extends Foo[Int]:
2   override def foo: Int
```

Listing 11: Generated specialized interface trait, after inline trait pruning

The compiler then replaces all references to `Foo[Int]` with `Foo$sp$Int`:⁵

```
1 def f(b: Foo$sp$Int): Int = 37 + b.foo
```

Listing 12: Interface replacement

Additionally, any anonymous class references to `new Foo[Int]() {}` are replaced with a *specialized trait implementation class* `Foo$impl$Int`. This is simply a class whose only parents are `Foo$sp$Int` and `Foo[Int]`, and which thus also benefits from specialization by inlining:

```
1 class Foo$impl$Int(x: Int) extends Foo$sp$Int, Foo[Int](this.x):
2   private val Foo$$x: Int = this.x
3   override def foo: Int = this.Foo$$x
```

Listing 13: Specialized trait implementation class

Putting these elements together, the code specializes as in Listings 14 and 15:

```
1 inline trait Foo[T: Specialized](x: T):
2   def foo = x
3
4 def f(b: Foo[Int]) = 37 + b.foo
5
6 @main def main(args: Array[String]): Unit =
7   val y = f(new Foo[Int](42) {})
```

Listing 14: Source code using specialized traits

⁵i.e., instead of *erasing* to `Foo`, `Foo[Int]` *erases* to `Foo$sp$Int`. This is sound under the invariant that $\forall T, T <: \text{Foo}[Int] \iff T <: \text{Foo\$sp\$Int}$

```

1 inline trait Foo[T: Specialized](x: T):
2   def foo = x
3
4 inline trait Foo$Sp$Int extends Foo[Int]:
5   override def foo: Int
6
7 class Foo$Impl$Int(x: Int) extends Foo$Sp$Int, Foo[Int](this.x):
8   private val Foo$$x: Int = this.x
9   override def foo: Int = this.Foo$$x
10
11 def f(b: Foo$Sp$Int): Int = 37 + b.foo
12
13 @main def main(args: String[]): Unit =
14   val y: Int = f(new Foo$Impl$Int(42))

```

Listing 15: Result of specialization

2.3 Benefits of Specialized Traits

This approach provides interface specialization, while avoiding the problems outlined in section 2.1:

- **Code Bloat:** The specialized instance classes and interface traits are generated by the compiler and so do not appear in the source. The only modification required to use specialization is to add the `Specialized` context bound, assuming the trait is already inline.
- **Lack of Standardisation:** As the generated specialized interfaces and implementations are produced by the compiler, and there is only one way to trigger this behaviour (with the `Specialized` context bound), they are standardised.
- **User Responsibility:** The creator of a given trait determines whether or not to apply the `Specialized` context bound. Once this is applied, the trait can only be used in specialized form. This shifts responsibility for the decision to specialize onto library authors, without nevertheless pre-generating any specializations. Library authors are better placed to decide whether specialization is appropriate for their traits.
- **Generic Specialized Methods:** As long as a method is defined inline, we will know the types supplied as type arguments to any specialized trait it may instantiate, so we can specialize these after inlining.

Thus we introduce *specialized methods*, inline methods with at least one type parameter having a `Specialized` context bound. Note, however, that unlike specialized methods in Scala 2, we do not generate multiple specialized versions of these, as their specialization happens upon inlining, and they do not typically exist at runtime. Listing 16 gives an example.

```

1 object FooFactory:
2   inline def apply[T: Specialized](x: T) = new Foo[T](x){}
3
4 val x = FooFactory.apply[String]("Hello")
5 // inlines to:
6 val x = new Foo[String]("Hello"){
7 // which we can specialize to:
8 val x = new Foo$impl$string("Hello")
9
10 // In the case of the times method from before:
11 inline def times[T](x: Vec[T], y: Vec[T])
12 // the inlined code will be typed as if we had:
13 inline def times[Int](x: Vec[Int], y: Vec[Int])
14 // which we can now specialize to:
15 inline def times(x: Vec$sp$Int, y: Vec$sp$Int)
16 // and use values of type Vec$sp$Int for x and y

```

Listing 16: Specialized methods

3 Behaviour, Rules & Motivation

In this section we go into further detail on the behaviour of the specialized traits feature and its relation to other components of the language, and motivate the design decisions relating to the specialization API.

We discuss the following points:

- Rules for creating instances of specialized traits
- Specialization to reference types
- Families of specialized traits and partial specialization
- Transportation of `Specialized` through generic code

Design considerations related to the implementation of the feature are discussed in section 5.

3.1 Creating Specialized Trait Instances

3.1.1 Anonymous Class Instances & Implementation Classes

As exemplified in Listing 15, specialized traits can be instantiated via anonymous class instances `new Foo[Ts](ps) {}` which are replaced by references to specialized implementation classes `Foo$impl$Ts(ps)`. The use of such implementation classes has a few benefits:

- **No Manual Specialization:** Allows instance creation for specialized traits without having to manually inherit from the traits (which would recreate the issue of manual specialization).
- **Fewer Class Definitions:** Sharing `$impl$` classes between uses avoids bytecode explosion due to a large number of anonymous class definitions which would otherwise be generated once per use.
- **No Anonymous Classes in Bytecode:** Enables the compiler to reason that anonymous classes extending specialized traits cannot exist after specialization has been applied, (as they must have been replaced by the implementation classes) and this makes cycle checking for inlining easier.

However, in order to support reuse of the implementation classes, we must subject anonymous class instances of specialized traits to some restrictions: [6]

- can extend only a single specialized trait
- cannot mix in further classes or traits
- cannot contain member definitions.

These restrictions ensure that all anonymous class instances of the same specialized trait are the same, and so they can use the same implementation class; none of them can have extra methods or mixins that would break the reuse.

3.1.2 Direct Inheritance

We also allow ordinary classes and objects to inherit from specialized traits. This allows bypassing of the above restrictions by naming a type, which is useful in situations like for typeclasses, where the methods can only be defined in the specialized child class, because they depend on the type supplied:

```
1 inline trait Numeric[T: Specialized]:
2   def plus(x: T, y: T): T
3   def times(x: T, y: T): T
4
5 implicit object IntIsIntegral extends Numeric[Int]:
6   override def plus(x: Int, y: Int): Int = x + y
7   override def times(x: Int, y: Int): Int = x * y
```

Listing 17: Named object extends specialized trait

The methods `plus` and `times` depend on the type `Int` for their definition here, so they cannot live in the inline trait. Due to the restriction that anonymous instances of specialized traits may not contain method definitions, it would therefore be impossible to implement the instances of this specialized `Numeric` typeclass through only anonymous instances of specialized traits.

Erasure will replace the reference `Numeric[Int]` in the parents of `IntIsIntegral` with the specialized interface `Numeric$sp$Int` which will allow specialized access to the methods of `IntIsIntegral` via this interface.

While `objects` and `classes` may extend specialized traits, we do not allow `traits` to extend specialized traits, unless the trait is inline. This is due to the restriction on extending inline traits with parameters by ordinary traits, explained in section 4.5, since use of the `Specialized` typeclass implies an evidence parameter.

3.2 Reference Types, Specializing Supertype & Variance

As mentioned in section 1.2, the primary use of specialization is to remove the need for boxing around primitive types, however there is a benefit to allowing specialization to object types as well.

While reference types are not *boxed*, erasure means they must be treated as `Object` when used to fill generic parameters. This means that a downcast (e.g. `x.asInstanceOf[String]`) must be inserted by the compiler when a generic member of reference type (`x: T` erased to `x: Object`) is to be used (for example to call a method on it). On the JVM such casts are *checked* for validity so that an exception can be thrown if the cast is not viable, and this checking imposes some overhead.

Furthermore, monomorphic calls on concrete reference types can sometimes be *de-virtualised* by the JIT, which makes them faster than the same calls on generic reference types. This makes specialization to reference types interesting. An example of this is illustrated in section 6.1, in the `PairHashing` benchmark.

However, specializing to any type runs into two potential issues:

- **Type Parameters:** Reference types may take type parameters, for example `Queue[T]`. Specializing `Foo[Queue[Int]]` and `Foo[Queue[Boolean]]` separately would:
 - multiply the number of potential generated specializations
 - not be beneficial unless `Queue[T]` was also specialized, because erasure of the generated specialized trait interface classes would make them identical (any reference to `Queue[Int]` in `Foo$sp$QueueInt` would become the generic `Queue`)
 - lead to increased complexity
- **Variance:** Furthermore, type parameters may be variant in Scala. This means that container types can inherit subtyping relationships from their type variables. For example:
 - `List[+T]` means that `List[Dog] <: List[Animal]` because `Dog <: Animal`.⁶
 - `RecyclingBin[-T]` means that `RecyclingBin[Paper] <: RecyclingBin[Newspaper]` because `Newspaper <: Paper`

These relationships exist at compile time in the typer, but do not require any special runtime implementation. This is possible because of erasure, since the runtime representation of `List[Dog]` is the same as the runtime representation of `List[Animal]`, namely just `List`.

It is the typecasts applied to values extracted from the `List` that apply type `Dog` or `Animal` to these values. Of course, these casts should only be applied where they cannot fail, but this has already been checked by the typer.

The situation for specialized types is different, because they truly have a different runtime representation. If we specialize `List[Dog]` to `List$sp$Dog` and `List[Animal]` to `List$sp$Animal`, we cannot automatically use `List$sp$Dog` in the place of `List$sp$Animal`.

It is conceivable to want to manually wire the subtyping relationships together by having `List$sp$Dog` extends `List$sp$Animal` for example, but this quickly becomes unmanageable, especially in the contravariance case: `RecyclingBin$sp$Paper` must extend `RecyclingBin$sp$T` for all `T <: Paper`. There may be arbitrarily many of these traits and they may not be known a priori if `Paper` is not `sealed`.

⁶The notation `A <: B` means that `A` is a subtype of `B` or “an `A` is-a `B`”.

For these reasons we introduce the notion of a specializing supertype [6] and only specialize to that type:

- A *simple class type* is a reference to a static class that does not have type parameters. References to traits and references containing non-static prefixes or refinements are excluded.
- A *top class* is one of `Any`, `AnyVal`, `Object` or `AnyRef` (synonym of `Object`)
- The *specializing supertype* of a type `Tp` is defined as follows:
 - `SpecType(Tp)` is the smallest⁷ simple class type `C` such that:
 - * `Tp <: C`
 - * The superclass of `C` is a top class, or `C` is itself a topclass
 - `SpecType(Nothing) = Nothing`

If a type `Tp` is used to fill a specialized parameter slot in a specialized trait `Foo`, the trait erases to:

- Its normal erasure `Foo` if `SpecType(Tp)` is one of the top classes or `Nothing`. In this case there is no specialization.
- `Foo$sp$SpecType(Tp)` otherwise.

`$impl$` classes are generated by the same rule (with `$impl$` instead of `$sp$`). However, an anonymous class extending a specialized trait with only top class and/or `Nothing` arguments is converted to an instance of a special implementation class `Foo$impl`. This ensures we always have an implementation class no matter the type arguments, so anonymous class instances are always replaced.

The `SpecType` approach means that, for example, `List[Dog]` and `List[Animal]` both specialize to `List$sp$Animal` (unless `Animal` has other superclasses) and so the variance is implementable. In addition, the definition of *simple class type* excludes types with type parameters, which resolves the first issue.

However, while the variance issue is resolved for specialization to user-defined reference types, the problem remains for types that sit in different “buckets” of the `SpecType` function and therefore erase differently. In particular, the following variance patterns are forbidden if `List` and `RecyclingBin` are covariant and contravariant specialized traits respectively:

- *Contravariance with top classes (value types)*: `RecyclingBin[Any]`, `RecyclingBin[AnyVal]` erase to `RecyclingBin` and thus may not be passed to `RecyclingBin[Int]`, which erases to `RecyclingBin$sp$Int`, whereas normally this would be allowed.

⁷In the sense of “smallest by set of values that the type contains”, i.e. closest to the leaves of the type hierarchy.

- *Contravariance with top classes (reference types)*: `RecyclingBin[Any]`, `RecyclingBin[Object / AnyRef]` may not be passed to `RecyclingBin[Paper]` whereas normally this would be allowed
- *Covariance (Nothing)*: `List[Nothing]` may not be passed to `List[T]`.

In contrast, `RecyclingBin[Any]` may be passed to `RecyclingBin[Object]`, `RecyclingBin[AnyRef]`, `RecyclingBin[AnyVal]` as these all erase to `RecyclingBin`.

This restriction on variance is reasonable since it originates directly in the dichotomy between the generic erased representation and the specialized versions. The user requested specialization, which is simply at odds with the full variance pattern implemented via erasure.

Nonetheless, as we discuss in section 7, this approach greatly limits the potential for speed-up via specialization to reference types. An alternative would be to abandon variance for specialized traits entirely, and thus be able to specialize to every user-defined reference type.

3.3 Families of Specialized Traits & Partial Specialization

Specialized traits may extend other specialized traits. In this case the generated child interface trait extends the specialized interface traits of all parent traits of the original specialized trait.

```
1 inline trait Vec[T: Specialized] extends Seq[T]
2 inline trait Vec$sp$Int extends Vec[Int], Seq$sp$Int
```

Listing 18: Extending another specialized trait

This enables the specialized trait to be used in contexts expecting:

- The specialized trait `Vec$sp$Int` itself (i.e. `Vec[Int]` in source code)
- A generic `Vec` (i.e. `Vec[?]` in source code, although in the current implementation this is forbidden)
- Specialized traits higher in the specialized hierarchy such as `Seq$sp$Int`.

Furthermore, families of specialized traits with multiple `Specialized` parameters may be *partially specialized*, and this works because of the requirement to transport `Specialized`, which is discussed in the following section.

```
1 inline trait Foo[T: Specialized, S: Specialized]
2 inline trait Bar[T: Specialized] extends Foo[T, Int]
3 class Baz extends Bar[Boolean]
```

Listing 19: Partial specialization

Only once `Baz` is defined to extend `Bar[Boolean]` do we have concrete types for all of the specialized type parameters. At this point the corresponding specializations can be generated:

```
1 inline trait Foo$sp$Boolean$Int extends Foo[Boolean, Int]
2 inline trait Bar$sp$Boolean extends Bar[Int], Foo$sp$Boolean$Int
3 class Baz extends Foo, Bar, Foo$sp$Boolean$Int, Bar$sp$Boolean
```

3.4 Transportation of Specialized through Generic Code

We require that a type argument used to instantiate a specialized trait or method must be one of:

- A fully concrete type
- A type whose only type variables are also marked with `Specialized`

In this way, we can build arbitrarily long chains of specialized method calls ending in instantiation of specialized traits, knowing that specialization flows correctly through the polymorphic specialized methods, and we always know the concrete types whenever a specialized trait is referenced (after inlining).

For example, in Listing 20 a call to `inlineMethod1[Int]` will inline to `inlineMethod2[Int]` which inlines to `new T2[Int]() {}`, leading to generation of `T2$impl$Int`, `T2$sp$Int` and `T1$sp$Int`.

```
1 inline trait T1[T: Specialized]
2 inline trait T2[T: Specialized] extends T1[T]
3
4 inline def inlineMethod1[T: Specialized] = inlineMethod2
5 inline def inlineMethod2[T: Specialized] = new T2[T]() {}
```

Listing 20: Transportation of Specialized through generic code

Perhaps surprisingly, this transportation rule is not inherent through the use of a context bound to annotate specialized parameters. In particular, the code in Listing 21 is valid. The implicit `Numeric` constraint on `S` is checked only when a concrete type is provided to `S`, which causes `T2` to instantiate its parent trait `T1` with the concrete `S` type in place of `T`.

```
1 inline trait T1[T: Numeric]
2 inline trait T2[S] extends T1[S]
```

Listing 21: Transportation of Numeric through generic code

In contrast, the pattern in Listing 21 is rejected with the `Specialized` context bound instead of `Numeric`, because it would lead to confusion over whether `T2` should be specialized or not.

4 Modifications to Inline Traits

The project began with an almost complete prototype of inline traits from [1]. A number of modifications were made to the semantics and implementation to:

- sharpen the semantics to clarify what is and isn't allowed, and ensure this is reflected in the implementation
- modify the behaviour where necessary to support specialized traits
- support some additional features which are required to get the most benefit out of inline and specialized traits
- fix some known bugs

In this section we detail the modifications made, with reference to section 5 of [1] where appropriate.

4.1 Replacement of Inline Trait Symbols

In sections 5.1 (Symbols), and 5.2.1, Andres explains the desire to replace references to members living in inline traits accessed on children of inline traits with the corresponding inlined symbols. He states that we need to do this throughout the entire compilation unit, rather than merely inside the body of classes extending inline traits. For example, observe Listing 22 adapted from Listing 7:

```
1 inline trait Foo:
2   def foo#1(): Object
3
4 class Bar extends Foo[Int](42):
5   override def foo#2(): Int = 42
6   override def foo#3(): Object = Int.box(this.foo())
7
8 def f(b: Bar): Int = 37 + Int.unbox(b.foo#1())
```

Listing 22: Member replacement is required

Even though `b: Bar` is typed with the child class type, without additional work the call to `b.foo` will still reference `foo#1` because the symbol was resolved before inline trait inlining, and so we have no specialization. The desire is thus to replace calls to `foo#1` with calls to `foo#2` by retyping them to use the specialized version. In this way there would be no boxing generated around the call.

While Andres suggests a separate phase *postSpecializeInlineTraits* to perform this operation, we implement it in the *erasure* phase. The motivation for this is to coordinate with specialized traits as follows:

- We need to see the erased `A$sp$Int` traits so we can point at their members and gain specialization.

- We need to make the replacement before boxing to avoid incorrect typing: if `Foo.foo` is typed with `T`, and `Bar.foo` specializes this to `Int`, as above, performing the replacement after boxing would lead to `Int.unbox(B.foo#2)` instead of directly `B.foo#2`, and this doesn't typecheck.

The first condition implies a lower bound of *erasure* on the phase that we perform the replacement in, and the second condition implies an upper bound of *erasure*. It turns out that it is possible within the *erasure* phase to meet both of the constraints.

4.2 Additional Language Features in Inline Traits

We add support for:

- Super calls into & from inline traits
- Opaque types as members of inline traits
- Self types for inline traits

4.2.1 Super Calls

Super Calls into Inline Traits

Consider the example in Listing 23:

```

1 inline trait A:
2   def foo = "Hello World"
3
4 class C extends A:
5   override def foo = super[A].foo

```

Listing 23: Super calls into inline traits

As inline traits are converted to pure interfaces it is not possible to make a direct super call to the method `A.foo` as it is abstract after pruning. Furthermore even if this were possible, specialization would still be lost.

Therefore, overridden methods which are referenced by super calls into inline traits are inlined into the inline receiver with a mangled name, e.g. `A$$foo` which allows us to transform super calls to refer to these directly.

In contrast, methods that are not overridden in the child class do not need to be mangled. Since these methods are inlined normally and we know that the method has the same definition as it did in the parent (as the parent is an inline trait), we can simply transform the super call to a `this` call of the same name.

Super Calls from Inline Traits

However, we impose one restriction on `super` calls made *in* inline traits: Inline

traits may not contain super references to classes or non-inline traits.

This is because super references in Scala may only reference direct parents, and, after inlining, those references that were to direct parents in the original inline trait would now have to point to ancestor classes which are further than 1 hop away from the inline receiver class, which is impossible.

As noted above, we cannot call into the inline trait and have the inline trait call the grandparent because the method in the inline trait will be abstract after pruning and so cannot be called.

`Super` calls from inline traits into other inline traits *are* allowed however, because the grandparent's method will be inlined into the parent inline trait and again into the child inline trait, because of the behaviour change discussed in section 4.3.2. Listings 24 and 25 illustrate this:

```
1 inline trait A:
2   def foo = "A"
3
4 inline trait B extends A:
5   override def foo = super.foo
6
7 class C extends B:
8   override def foo = super.foo
```

Listing 24: Super calls from inline traits

```
1 inline trait A:
2   def foo: String = "A"
3
4 inline trait B extends A:
5   private[this] def A$$foo: String = "A"
6   override def foo: String = this.A$$foo
7
8 class C extends B:
9   private[this] def A$$foo: String = "A"
10  private[this] def B$$foo: String = this.A$$foo
11  override def foo: String = this.B$$foo
```

Listing 25: Super calls from inline traits, after compilation

4.2.2 Opaque Types

Opaque types are now supported in inline traits, with the same semantics as in ordinary traits. It is not allowed to use inlining to ‘cheat’ opaque types, as illustrated in Listing 26, even though it is tempting to think that since the opaque type will be inlined into B it should be visible.

```

1 inline trait A[T](val x: T):
2   opaque type Special = T
3
4 class B extends A[Int](100):
5   def foo: Special = 10 // error: 10 doesn't conform to Special

```

Listing 26: Opaque types in inline traits

4.2.3 Self Types

Self types are supported but are not inlined. We argue this is desirable as it ensures that the behaviour of inline traits with self types mirrors that of ordinary traits with self types.

Inlining of self types would effectively remove any restrictions that these self types seek to impose because the subclass would automatically have a matching self type to that of the parent class. This would prevent an error from being issued, irrespective of whether the subclass implements the desired traits.

Therefore, when using self types on inline traits, as shown in Listing 27, the behaviour observed is the same as in ordinary traits:

```

1 inline trait A[T]:
2   this: T1 =>
3
4 class D extends A[Int] // error: self type of D does not conform to T1

```

Listing 27: Self types in inline traits

4.3 Inlining Before Pickling & into Inline Traits

This section discusses perhaps the two most significant modifications to the behaviour of the inline traits implementation:

- Moving of the *specializeInlineTraits* phase to be before the *pickling* phase in the compiler.
- Performing inlining into inline traits which extend other inline traits.

4.3.1 Inlining Before Pickling

Pickling is the phase at which the Scala compiler writes the typed abstract syntax tree of the program to TASTy files. These are used to read the interface of types referenced across compilation units in separate compilation, to support type checking.

For example, if `class B` in `B.scala` references a method on `class A` in `A.scala` which has already been compiled, the later compilation of `B.scala` will cause the compiler to read in `A.tasty` to access the interface of `class A`'s methods.

It was observed that the previous placement of the *specializeInlineTraits* phase, after pickling, is problematic when inline trait child classes are referenced across compilation units. Consider the example in Listing 28:

```
1 // A.scala
2 inline trait Foo[T]:
3   def foo: T
4
5 class Bar extends Foo[Int]
6
7 // B.scala
8 def f(b: Bar): Int = 37 + b.foo
```

Listing 28: Inline trait reference across compilation units

When we compile `B.scala` we see the definition of `class Bar` as it was at the pickling phase, because this information is read from `Bar.tasty`. This means that we do not see the inlined method `def foo: Int` in `class Bar`, so we assume the reference to `b.foo` must point to the inherited `foo` in `Foo[T]`. Thus we get no specialization.

In contrast, if we perform inlining before pickling, the compilation of `B.scala` will see the specialized version `def foo: Int` inlined into `class Bar` when reading `Bar.tasty`. In this way, specialization is preserved across compilation units as the reference `b.foo` on line 8 will point to the specialized version.

4.3.2 Inlining into Inline Traits

As mentioned in section 2.2.2 of [1], the original implementation of inline traits only inlined members into the first non-inline descendant of an inline trait. This is illustrated in Listing 29.

```
1 inline trait Foo[T]:
2   def foo(x: T): T = x
3 inline trait Bar extends Foo[Int]
4 class Baz extends Bar
5
6 // inlines to:
7 inline trait Foo[T]:
8   def foo(x: T): T
9 inline trait Bar extends Foo[Int]
10 class Baz extends Bar:
11   def foo(x: Int): Int = x
```

Listing 29: Inlining into the first non-inline descendant

Since inline traits are pruned, the only difference between this and inlining into every direct child of every inline trait is that, with this approach, inline traits do not gain interfaces from their parent inline traits. In Listing 29 this means that `Bar` has no `foo` method.

As the original idea with inline traits was that members should not be accessed via inline traits (as this loses specialization) this was insignificant. However, when we introduce specialized traits this configuration becomes problematic.

We desire that the generated `$sp$` traits for specialization be inline for a few reasons, which are discussed in the following section. Furthermore, it is critical that the `$sp$` traits contain the specialized interfaces of the original specialized trait, as we intend for specialized access to be through this interface. These two observations imply that we must change the semantics of inline trait inlining to always perform inlining of inline traits into children of these inline traits, whether these children are themselves inline traits or not.

In this way, `inline trait Bar` in Listing 30 also gets a copy of the interface `def foo(x: Int): Int` from `Foo[Int]`.

```
1 inline trait Foo[T]:  
2   def foo(x: T): T  
3 inline trait Bar extends Foo[Int]:  
4   override def foo(x: Int): Int  
5 class Baz extends Bar:  
6   override def foo(x: Int): Int = x
```

Listing 30: Inlining into all children

This does increase the number of interface methods we need to keep track of, but this is unlikely to be significant in terms of generated code size and is a worthy trade-off for implementing specialized traits.

4.3.3 Desire for Specialized Interface Traits to be Inline

We require the generated `$sp$` traits to be inline for the following considerations:

- **Confusion when Adding Specialized:**
Consider the difference between Listings 31 and 32:

```

1 inline trait A[T]:
2   def foo = "Hello World"
3 inline trait B extends A[Int]
4 class C extends B
5 // inlines to:
6 inline trait A[T]:
7   def foo: String
8 inline trait B extends A[Int]:
9   def foo: String
10 class C extends B:
11   def foo = "Hello World"

```

Listing 31: No Specialized

```

1 inline trait A[T: Specialized]:
2   def foo = "Hello World"
3 inline trait B extends A[Int]
4 class C extends B
5 // would expand to:
6 trait A$sp$Int extends A[Int]:
7   def foo = "Hello World"
8 inline trait B extends A$sp$Int
9 class C extends B

```

Listing 32: With Specialized

When `A$sp$Int` is not inline, merely adding `Specialized` changes the final location of the `foo` method, which we find confusing since the `$sp$` traits do not exist in the source code. Everything in the source code would seem to imply that the `foo` method would travel all the way down to end up in `class C`, and that is not the case if `A$sp$Int` is not inline.

- **Violates Other Rules:** Ordinary traits are not allowed to extend specialized traits. It therefore seems less than ideal to rely on generating ordinary traits that extend specialized traits to implement specialization.

4.4 Inline Trait Cycle Checking

We implement cycle checking for inline trait inlining. This means that code such as that depicted in Listing 33 throws an error instead of crashing the compiler.

```
1 inline trait C[S]:  
2   def v(x: S): S = x  
3   def w: Unit =  
4     val x = new D[S] {}  
5  
6 inline trait D[S]:  
7   def v(x: S): S = x  
8   def w: Unit =  
9     val x = new C[S] {}  
10  
11 def main =  
12   val x = new C[Int] {} // error: Inlining of inline traits looped.
```

Listing 33: Inline trait cycle detection

For this purpose we introduce `InlineTraitState` carried by the `Context` in the compiler, which stores a map `inlineOrigins` from inlined symbols to the trait that the symbol was inlined from. This enables cycle tracking even though inline trait inlining is not directly recursive in `Inlines.inlineParentInlineTraits`.

We have a cycle if either of the following conditions are true:

- We try to inline an inline trait into the definition tree of a symbol whose owner chain contains that same trait.
- We try to inline an inline trait into the definition tree of a symbol whose inline origins chain contains that same trait.

The latter condition is necessary in situations like the one depicted in Listing 33 on line 12, where `C` is inlined into `new C[Int] {}`, and then `D` is inlined into the inlined body, and `C` is then inlined into that generated body and so forth. In this case there is no inlining of `C` into `C` or `D` into `D` directly.

The former condition picks up a subset of the cycles picked up by the latter condition, but is also necessary to pick these cycles up as early as possible, before they crash the inliner, as the inliner does not support moving a symbol into its own owner chain.

Note that in the example in Listing 33 we would not issue an error without the reference to `C[Int]` on line 12. This is because, although there is a cycle due to inlining of the body of `C` into the anonymous class on line 9, this would later be pruned out during `pruneInlineTraits`, so treating the compilation as atomic this cycle does not exist from the point of view of the programmer.

Adopting this view additionally makes the rules surrounding inline trait inlining easier, as we can say that inline traits are never inlined into classes whose owner chain contains an inline trait (because we know such classes must exist on the right hand side of inline trait methods, and therefore be pruned). This approach avoids a number of edge cases.

4.5 Sharpening of Semantics

Furthermore, we sharpen the semantics of inline traits so it is clear what is and is not supported, and add enforcement to those cases which were illegal but not rejected previously. We make two particular precisions:

- Traits are allowed to extend inline traits if and only if the inline trait has no value parameters. This is because otherwise the value parameter fields risk becoming undefined due to the rules of trait parameter passing:

```

1 inline trait A[T](x: T):
2   val y = x
3 trait B extends A[Int]
4 class C extends A[Int](10), B

```

Listing 34: Inline trait inlining with parameters risks being undefined

After inlining, `B.y` is defined in terms of `B.A$$x` (the inlined copy of the parameter accessor of `x`), but this is undefined as we don't have the parameter value in `B` due to the trait parameter passing rules that insist that `C` passes the parameters directly (traits may not pass parameters to each other).

In `C` this is not a problem as we have the parameter value `10`. Even if we try to provide the value by leaving `A$$x` abstract and overriding in `C`, this will not work as `B.A$$x` will still be undefined.

Therefore, we reject this pattern. Note that this restriction does not block the reverse pattern *inline trait extends trait*.

- We modify some of the rules around overrides and conflicting members in order to make the behaviour more consistent with ordinary traits. In particular, we require the `override` modifier to resolve conflicts in a number of locations where previously conflicts were resolved on the basis of “last extending trait wins”. Some cases which cannot be supported due to potential name conflicts are rejected. The full details are described in the inline trait feature documentation, which can be found in the specialized traits PR.

4.6 Bug Fixes

We also address a number of bugs in the original implementation, including those elaborated on in section 5.1 of [1]. Among others, we address the following issues:

- *Private members are not renamed, only private parameter accessors are* (5.1) - this seems not to be an issue. It appears this was already fixed after Andres' thesis went to print, and therefore no changes are made.
- *Inner class-likes cause issues* (5.1) - we reject inner class-likes inside inline traits, as these are not needed for a first version, and the existing implementation was not complete.
- We apply the phase *pruneInlineTraits* slightly earlier than in the original implementation to avoid spurious warnings with `-Wsafe-init`.
- We fix flag copying in inline trait inlining, and resolve issues with inlining of inline methods into inline traits.

4.7 Remaining Future Work for Inline Traits

The inline traits feature is relatively reliable (for an unreleased feature!) as it has a large corpus of tests from [1], and is depended on by specialized traits which are also reasonably well tested. However, towards the end of the project a few additional issues were identified which need to be resolved before the feature can be used in earnest:

In Listing 35, the method `problem` should not be inlined into `class Baz`. This is because the inlining should stop at `Bar` which is not inline. This is a relic of the former behaviour which inlined at the first non-inline class, and the previous ambiguity over whether traits should be allowed to extend inline traits, but this is easily fixable.

```
1 inline trait Foo:
2   def problem = "Hello World"
3 trait Bar extends Foo
4 inline trait Confounder
5 class Baz extends Bar, Confounder
```

Listing 35: Inline trait inlining is not blocked

This issue can be found in test `inline-trait-non-inline-blocks-inlining.scala` in the PR.

Trait extends inline trait can sometimes cause a compiler crash. This issue is found in `tests/run/trait-extends-inline-trait-uses.scala`.

There are also issues with using `ClassTag` parameters on inline traits, and spurious warnings on some local private members.

Beyond these issues, future work on inline traits may wish to tackle the following points, many of which were also listed as future work in [1]. These were not prioritised for this project as the focus lay with specialized traits:

- Supporting inner classes and/or traits inside inline traits. This is relatively complex, as a number of challenging situations can arise, such as `inline-trait-double-nested-class.scala`. A large body of tests for inner classes can be found both in `tests/neg` and `tests/disabled` in the PR.
- Supporting inline parameters in inline traits, similar to how inline methods may take inline parameters. This could be of interest to improve the utility of `inline vals` inside inline traits.
- Pruning some more useless fields in inline traits, as suggested in section 5.2.3 of [1].
- Printing a warning if specialization is lost due to member access through an inline trait which is not a specialized interface trait, and advising the user to apply specialization. (section 5.2.4 - User experience of [1]).
- Implementing postphase checks (section 5.3.1 of [1]).

5 Implementation

The specialized traits feature has been implemented in the Scala 3 compiler; the code for it is to be found in [Pull Request#26156](#). In this section we discuss this implementation of the feature and the associated tradeoffs. The principal changes for this feature are:

- Creation of a new miniphase *desugarSpecializedTraits*
- Modifications to *erasure* to erase specialized traits correctly
- Creation of a new miniphase *pruneSpecializedMethods*

These are discussed in the following sections.

5.1 *DesugarSpecializedTraits*

The new miniphase *desugarSpecializedTraits* performs the bulk of the work for the feature:

- Enforces rules of specialized traits, including:
 - Transportation of **Specialized** through generic code
 - Limitations on anonymous class instances extending specialized traits.
 - Only inline traits and inline methods may take **Specialized** type parameters.
 - Wildcard types may not be substituted for **Specialized** type params.
- Scans the code for specializations of specialized traits.
- Generates required **\$impl\$** classes and **\$sp\$** traits based on referenced specializations.
- Inlines specialized methods.

5.1.1 Enforcement of Specialized Trait Rules

The rules of specialized traits are mostly discussed elsewhere in this document, with the exception of the rejection of wildcard (?) types used as **Specialized** type arguments.

Initially it was thought that these would be challenging to support, because it would require bridge methods that “despecialize” to ensure that e.g. **A\$sp\$Int** implements **A**’s interface. In the end these bridge methods have been successfully implemented, as discussed in section 5.2, so it should be very possible to lift this restriction as future work. This would be desirable to support methods such as the following:

```

1 extension (vec: Vec[?])
2   def size: Int = vec.length

```

Listing 36: Use of question mark

5.1.2 Collection of Referenced Specializations

This occurs in the function `collectReferencedSpecializations`. Specializations are identified by pattern matching on the `Specialization` object, whose `unapply` method identifies type and tree patterns which correspond to an application of a class/trait to type arguments.

It also contains a number of methods to conveniently work with these arguments, including identifying which correspond to specialized parameters, and linking arguments, type members and constructor type parameters of the trait.

Once a specialization has been identified, it is attributed a `$sp$` trait / `$impl$` class symbol and stored in the `SpecializedTraitCache`. This is a data structure which tracks the specializations found in the program, along with the interface trait and implementation class symbols that have been attributed to them. This ensures we only generate one instance of each specialization.

The `SpecializedTraitCache` has a particular model. It contains two ‘levels’ which support processing over multiple ‘iterations’:

- specializations and corresponding interface/implementation symbols that we have found since the last *installNewInterfaceSymbols* / *installNewImplementationSymbols* call (i.e. on this iteration)
- those specializations that we found prior to that call, that were thus installed previously

The reason for this is that we need to be careful to correctly interleave the generation of specialized trait symbols and the inlining of their bodies:

```

1 inline trait C[S: Specialized]
2 inline trait A[T: Specialized]:
3   def x(y: C[T])
4 class B extends A[Char]

```

Listing 37: Repeated inlining and specialization steps are required

In Listing 37, the reference to `A[Char]` in the parents of `class B` demands the creation of `A$sp$Char`. After inlining the body of `A` into `A$sp$Char`, we have the method `def x(y: C[Char])`, which implies we need to generate `C$sp$Char`. This repetition can continue for arbitrarily many steps and may also involve implementation classes.

Our original idea for specialized traits was to generate shells for all of the interface traits and implementation classes in the new phase, and then rely on the existing *specializeInlineTraits* phase to perform the inlining. Clearly this example shows that such an approach is not possible as it would require alternating the phases in a loop.

For this reason we assign responsibility for inlining of inline traits in the following way if **A extends B** and B is an inline trait:

- If A is a specialized trait interface trait or implementation class, inlining happens in *desugarSpecializedTraits*.
- Otherwise, inlining happens in *specializeInlineTraits*, irrespective of whether B is specialized or not.

This means that in *desugarSpecializedTraits* we need to recursively loop the following steps:

- collection of referenced specializations
- generation of the **\$sp\$** trait and **\$impl\$** class symbols and bodies
- inlining into the generated **\$sp\$** and **\$impl\$** bodies.

To avoid generating the same class multiple times, we need to track which specializations we have already generated and inlined children for (those which have been “installed”), and which ones have been detected after the latest round of inlining and therefore still need body generation and inlining. The **SpecializedTraitCache** model takes care of this.

It would also likely be possible to implement this iterative inlining and specialization generation more directly by having **collectReferencedSpecializations** return a list of the specializations it found on this iteration, and then combining that with the previously found specializations.

The current design encapsulates this inside an object. It is partly the way it is since it is the result of a previous iteration of the design which performed direct replacement of specializations by the interface traits during the *desugarSpecializedTraits* phase.⁸, however it does have two advantages:

- As the name suggests, if caching of specialized traits is implemented (see section 7) it may be possible to combine that with the existing **SpecializedTraitCache** to make this easier.
- The **SpecializedTraitCache** correctly handles the required behaviour that “implementations imply interfaces” (if we generate **Foo\$impl\$A** we must also generate **Foo\$sp\$A**), as well as duplicate management, and encapsulates some of this from class clients.

⁸This turned out to be undesirable for the reasons discussed in section 5.2.

5.1.3 Implementation Class and Interface Trait Generation

As mentioned in section 5.1.2, symbols are created for the generated `$sp$` traits and `$impl$` classes as soon as the specializations are identified. This happens via the functions `newInterfaceTrait` and `newImplementationClass`. These construct the correct parents, type parameters and flag sets for the generated traits and classes. The function `newInterfaceTrait` also checks if the specialization requires us to specialize any parents of the original specialized trait.

We need to be careful to generate the correct type parameters in the case of specialized traits that have other, non-specialized type arguments. In that case the generated `$sp$` traits and `$impl$` classes may need to inherit the type parameters:

```
1 inline trait A[T, D: Specialized]
2 class C extends A[Int, Char]
3
4 // leads to generation of
5 inline trait A$sp$Char[T] extends A[T, Char]
```

Listing 38: Inheriting of type parameters from specialized traits

This is important for effective reuse of `A$sp$Char` in situations where the `T` type may vary. In this process we run into a cycle that in order to generate the type parameters we need the symbol of the new trait / class, but in order to produce the symbol we need the parent types, which may rely on the type parameters.

For this reason we initially generate the traits / classes with type parameters copied from the parent inline trait, and then perform replacement by the newly generated type parameters after generating them.

The body trees for these classes are generated as skeletons by `buildInterfaceTraitTree` and `buildImplementationClassTree`, before inlining of inline traits (through the call to `Inlines.inlineParentInlineTraits`) fills them in.

It is important for disambiguation between specialized traits of the same name defined in different locations that we generate the `$sp$` traits and `$impl$` classes in the same package as the original specialized trait, however if this trait was pre-compiled and loaded from TASTy, we may not have a `PackageDef` for this package in the current tree. Therefore, we need to rearrange the `PackageDefs` at the top level of the current compilation unit tree to ensure we have one for each package that contains specialized traits. This is done when emitting the generated statements in the `transformUnit` function.

Furthermore, if the specialized trait was defined inside a class, trait or object we should also reflect this in the placement of the generated classes. However, this is challenging because unlike `PackageDefs` which may appear multiple times in

the same tree or across files and merely reflect the location of the generated code, class and trait definitions can only exist once in a single compilation unit and always lead to generation of a class file. Thus if the specialized trait in question was defined in another compilation unit, redefining the owner class or object in the current compilation unit may create duplicate class files or not be understood by build tools. It is also not particularly ideal to spontaneously decide to recompile classes that we are not supposed to be compiling.

For this reason, defining specialized traits inside traits and classes is prohibited in the first version of specialized traits. It is, however, allowed to define them inside standalone `objects`. This is because classes defined inside `objects` are later lifted by the rest of the compiler to the outermost scope. We can mimic this lifting for our generated classes and define them in that scope instead, without needing to redefine the object.

5.1.4 Inlining of Specialized Methods

There are two reasons to be concerned about inline methods when it comes to specialization:

- Specialized methods in which inlining causes specialization (reproduced from Listing 16):

```
1 inline def createFoo[T: Specialized](x: T) = new Foo[T](x){}
2 val x = createFoo[Int](10)
```

Listing 39: Inlining induces specialization

In this case we need to perform inlining before specialization to see the reference to `Foo[Int]` that we need to specialize. Note that it is not sufficient to observe the type given to the `createFoo` call. This is because the body of `createFoo` may contain `new Foo[T](x){}` without this being visible in the return type.

- Inline methods which inline pre-specialized instances, including when these are hidden. As in the above case, observing the types is insufficient, as shown in Listing 40:

```
1 inline def inlineMethod1 =
2   println("Before")
3   val y = new Foo[Int](10) {}
4   println("After")
5
6 @main def main = inlineMethod1
```

Listing 40: Inlining of a pre-specialized instance

The first case implies we need some form of method inlining before or during *desugarSpecializedTraits*. This exists in the form of `transparent inline` methods, but such methods have other side effects on typing and compatibility. To avoid having to insist upon `transparent inline` for specialized methods, we instead inline them ourselves in the *desugarSpecializedTraits* phase. Specialized methods are identified by `MethodSpecialization.unapply`, and then inlined before specialization.

In principle, since we only observe the inlined bodies in order to identify necessary specializations, and do not modify them, it would be possible to “undo” this inlining after generating the required specialized traits, and perform it for real at the usual *inlining* phase, knowing that the specializations needed for the inlined body are present. We do not attempt this.

The second case requires some thought, but turns out to work fine with the current setup. Assuming separate compilation, the compilation of the inline method will generate `Foo$impl$Int`, which will live in the same place in the build output as the `Foo` trait referenced in the source code.

Inlining will then read the TASTy file containing the inline method and inline it, all after *desugarSpecializedTraits* has already run in the destination compilation unit. This means that when *erasure* looks for the `Foo$impl$Int` class symbol to replace `new Foo[Int](10) {}`, the symbol will not have been generated in the current run. It will instead be read from the classpath and found at the location that it was left at when the inline method was compiled.

5.2 *Erasure*

As alluded to in section 2.2, the replacement of `Foo[Int]` with `Foo$sp$Int` and `new Foo[Int]() {}` with `Foo$impl$Int` takes place in the *erasure* phase.

This is required because the architecture of the Scala 3 compiler imposes that the signature of methods (essentially their erased types) may not change until at least the *erasure* phase.

However, since the symbols that will feature in the signature of methods with specialized trait parameters or return values (the `$sp$` traits) do not exist until the *desugarSpecializedTraits* phase, the signatures of such methods must start out unspecialized, as shown in Listing 41, which lower bounds the replacement at the *erasure* phase.

```

1 // Initially
2 inline trait Foo[T]
3 def foo(x: Foo[Int]): Foo[Int] = x // signature Foo -> Foo
4
5 // After desugar specialized traits
6 inline trait Foo$sp$Int extends Foo[Int]
7 def foo(x: Foo[Int]): Foo[Int] = x // may not modify signature until erasure
8
9 // After erasure
10 def foo(x: Foo$sp$Int): Foo$sp$Int = x // signature Foo$sp$Int -> Foo$sp$Int

```

Listing 41: Signature cannot be specialized initially

Initially it was attempted to perform this replacement earlier than *erasure* and not modify the signatures, but as well as being very fragile, this turns out to be incredibly challenging as it requires the construction of a `TreeTypeMap` which modifies symbol infos, and can cause issues with specialization of reference types.

The *erasure* phase thus gains four additional rules / transformation steps:

- Replacement of references to `Foo[Int]` with `Foo$sp$Int`.
- Replacement of references to `new Foo[Int]() {}` with `Foo$impl$Int`.
- Addition of parents to classes that manually extend specialized traits.
- Generation of *de-specializing bridges*.

The first two have been discussed at length, so we focus on justifying the remaining two.

5.2.1 Addition of Parents to Manually-Extending Classes

The effect of the third transformation is shown in Listing 42. It serves to maintain the invariant that $\forall T, T <: \text{Foo[Int]} \iff T <: \text{Foo\$sp\$Int}$.

```

1 class Bar extends Foo[Int](10)
2 // Becomes:
3 class Bar extends Foo(10), Foo$sp$Int

```

Listing 42: Addition of parents to manually extending classes

It is debatable whether the `Foo(10)` parent is strictly necessary here, given we know that `Foo` is an inline trait and therefore we don't really need to pass the argument to `Foo` in order to have it in `Bar` (as it has already been inlined), but it may be helpful for debugging to know which argument was used, and (although this is checked earlier in the compiler), we typically insist that `Bar` implements `Foo` directly since it implements `Foo$sp$Int` which extends `Foo`, and `Foo` takes parameters.

5.2.2 Bridge Methods

Erase is also the compiler phase responsible for generating bridge methods. These are any methods which are needed to make existing methods conform to erased interfaces. An example without specialization is shown in Listing 43.

```
1 // Before erasure:
2 trait Processor[T]:
3   def process(input: T): Any
4
5 class StringProcessor extends Processor[String]:
6   override def process(input: String): String = input.toUpperCase
7
8 // After erasure
9 trait Processor:
10  def process(input: Object): Object
11
12 class StringProcessor extends Processor:
13   override def process(input: String): String = input.toUpperCase()
14   override def process(input: Object): Object = this.process(input.asInstanceOf[String]) // bridge
```

Listing 43: Bridges in ordinary Scala code

We also need bridge methods to maintain interfaces after Specialization:

```
1 // Before erasure
2 inline trait Trait[T: Specialized]
3 inline trait A[T: Specialized]:
4   def foo: Trait[T]
5
6 class B extends A[Int]:
7   override def foo: Trait[Int] = new Trait[Int] {}
8
9 // After erasure
10 class B extends A, A$sp$Int:
11   override def foo#1: Trait$sp$Int = Trait$impl$Int()
12   override def foo#2: Trait = this.foo#1
```

Listing 44: Bridges for specialization

Bridge generation is naturally supported through existing infrastructure in *erasure*. This is another indication that placing specialized trait replacement in *erasure* is a convenient choice.

5.3 *PruneSpecializedMethods*

We add a further miniphase to run just before *pruneInlineTraits* which deletes all specialized methods. These would be deleted later anyway as they are non-retained inlines, but it was observed that if they are not removed before *pruneInlineTraits* they could cause compiler crashes due to dangling references, however this has not been able to be reproduced in the final version of the feature, so it is possible that this miniphase is no longer required.

6 Evaluation

6.1 Performance

6.1.1 Methodology

In this section we evaluate the generated code produced by the specialized traits feature across a number of JMH benchmarks, comparing it to a generic implementation and to manual specialization in each case. The code for these benchmarks can be found in Appendix A.

As observed in [2], to simulate a realistic workload it is important to test the generated code in both a monomorphic and a polymorphic environment. If only one type argument is ever used in place of a given generic parameter in the program, the JIT compiler can perform several monomorphisation optimisations which optimise the code at runtime, leading to a different picture of the performance of the generic code.

Like in that paper, we also limit ourselves to benchmarks with arrays, since the cost of pointer chasing in pointer-based data structures renders the impact of (even manual) specialization virtually negligible.

Furthermore we prioritise benchmarks in which the speed gain is a result of interface specialization due to specialized traits, rather than merely inline trait inlining. For example, in the `VectorDotProduct` benchmark the call to `scalarProduct(other: VecSpecialized[T]): T` is via the interface trait and so benefits from specialization.

We first provide a description of the benchmarks:

- **VectorDotProduct(n)**: A dot product is taken of two vectors of size n . In the monomorphic case we use $n = 10^8$, while in the polymorphic case we use $n = 10^7$. This enables us to get more representative results by allowing more executions of the benchmark in the 10 second time limit.
- **MatrixMultiplication**: Two square matrices of dimension 300×300 are multiplied together.
- **PairHashing**: An array of 5×10^6 pairs of values of reference type is hashed together using a naive xor of all the `hashCodes`. The intention is to show that reference types also benefit from specialization.

In each case, the monomorphic benchmark workload consists of a single invocation of the operation to a single type. The polymorphic benchmark workload consists of three invocations of the operation to distinct types.⁹

⁹Benchmarks are run on a Lenovo Ideapad Slim 3i running Debian 12 Bookworm with a 12th Generation Intel i5 processor and 16 GB of memory, with JDK 23.0.1 (HotSpot). We use 5 warm-up and 15 measurement iterations, each of length 10 seconds.

6.1.2 Results

Benchmark	Mono-/Polymorphic	Execution Time (ms) ¹⁰		
		Generic	Manual	Specialized
VectorDotProduct(10^8)	Monomorphic	470.1 ± 24.6	172.3 ± 0.6	172.1 ± 0.7
VectorDotProduct(10^7)	Polymorphic	420.0 ± 2.5	144.3 ± 1.5	149.6 ± 2.5
MatrixMultiplication	Monomorphic	101.6 ± 2.0	11.0 ± 0.2	10.8 ± 0.1
MatrixMultiplication	Polymorphic	1226.5 ± 5.1	33.5 ± 0.1	32.6 ± 0.1
PairHashing	Monomorphic	23.4 ± 1.4	23.1 ± 0.7	25.9 ± 0.8
PairHashing	Polymorphic	74.4 ± 0.5	38.8 ± 0.1	40.7 ± 0.6

Table 1: Benchmarks comparing generic, manually specialized and specialized trait implementations across mono- and polymorphic workloads

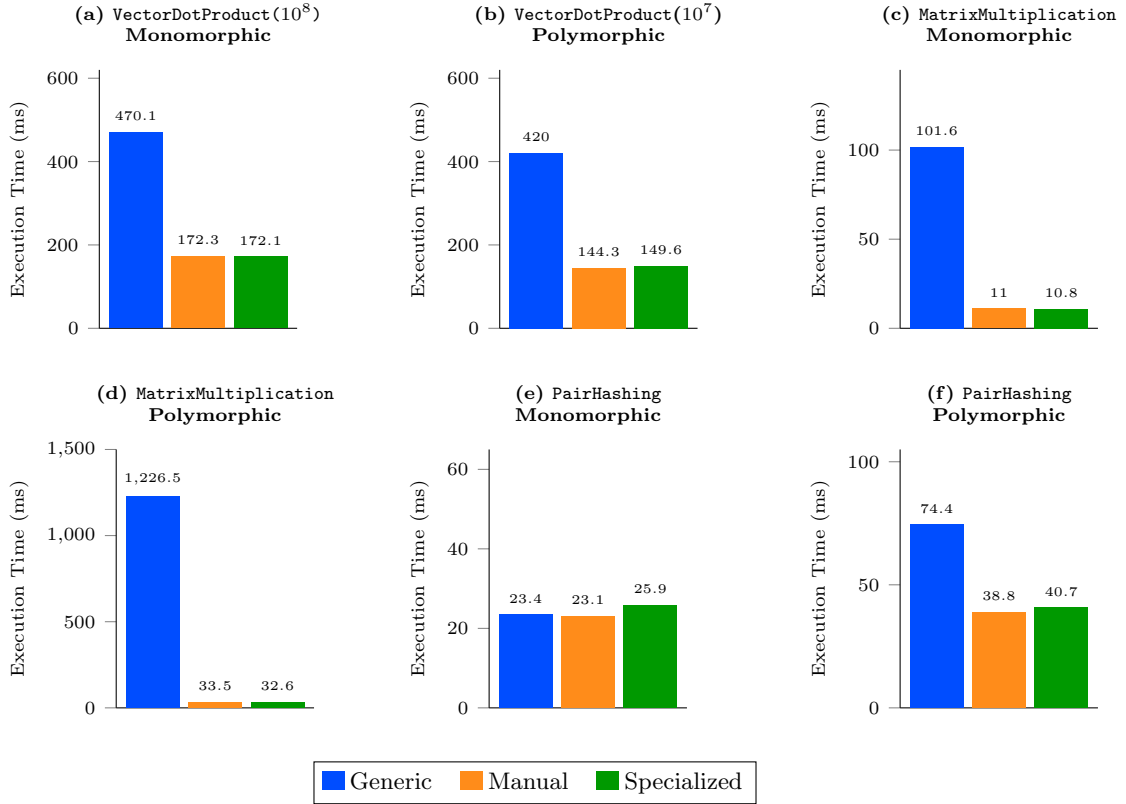


Figure 1: Benchmarks comparing generic, manually specialized and specialized trait implementations across mono- and polymorphic workloads.

In each of the benchmarks the specialized traits feature shows comparable performance with the manually specialized code, with an improvement on the generic code ranging from $2\times$ to $40\times$ depending on the benchmark in question. The following points are worth noting:

MatrixMultiplication

- The benefit of specialization is particularly pronounced in this benchmark, and that is likely because the matrices can fit in the CPU cache. This helps the specialized cases, which use primitive arrays, but penalises the generic implementation, which is forced to use an array of reference types.

PairHashing

- In this benchmark we don't have an interface to pass the specialized traits through, so in some respects the benefit here is exclusively due to inline traits. However, the advantage of specialized traits is that this sort of code can be written without a proliferation of anonymous class instances, thanks to the `$impl$` classes.
- Even with reference types we still see a $2\times$ improvement with specialization in the polymorphic case. This is because the calls to `x.hashCode()` and `y.hashCode()` are megamorphic in the generic pair case, which prevents the JIT from devirtualising them. In the specialized case these calls exist once per type combination, so the call sites are monomorphic and thus optimisable. In the monomorphic benchmark, all three implementations show comparable behaviour, because the JIT can optimise the monomorphic call sites in all of the implementations.
- **Limitation of this benchmark:** Because of the `SpecType` mechanism which only specializes to the topmost user-defined class in the hierarchy we can only show such a benefit with methods that are defined on generic `Objects`, such as `hashCode`. As discussed in section 7, specialization to other reference types may be desirable, but this would come at the cost of losing variance of specialized traits.

6.2 Generated Code

The generated code is in most cases functionally identical to the manually specialized code. Small disparities can be overcome by the JIT, however there are still some generated fields which are not necessary and could be removed, as discussed in [1].

Listings 45 and 46 compare the raw output after *erasure* of a vector class implemented respectively with manual specialization and with specialized traits.

¹⁰In each case we quote the execution time and the 99.9% confidence interval provided by JMH, rounded to 1 d.p. If the uncertainty falls below 0.05 we round up to 0.1.

The increased code length largely comes from the need to generate both `$sp$` traits and `$impl$` classes, and the various bridge methods required to maintain interface compatibility.

```

1 @SourceFile("tests/run/test1000.scala") @experimental("Added by -experimental")
2 class VecManual(elems: Int[]) extends Object() {
3   private val elems: Int[]
4   private val num: IntIsNumeric = IntIsNumeric
5   def length(): Int = VecManual.this.elems.length()
6   def apply(i: Int): Int = VecManual.this.elems.apply(i)
7   def scalarProduct(other: VecManual): Int =
8     {
9       require(this.length() == other.length())
10      var result: Int = this.num.fromInt(0)
11      scala.runtime.RichInt.until$extension(intWrapper(0), this.length()).
12        foreach((i: Int) =>
13          result =
14            this.num.plus(result, this.num.times(this.apply(i), other.apply(i)))
15        )
16      result: Int
17    }
18 }

```

Listing 45: Manually specialized class after erasure

```

1 @Child[VecSpecialized$impl$scala$Int] @SourceFile("tests/run/test1000.Scala")
2   @experimental("Added by -experimental")
3 inline trait VecSpecialized(
4   elems: Object, evidence$2: Numeric) extends Object {
5   private def elems(): Object
6   private given def evidence$2(): Numeric
7   def length(): Int
8   def apply(i: Int): Object
9   def scalarProduct(other: VecSpecialized): Object
10 }
11
12 inline trait VecSpecialized$$$sp$scala$Int() extends Object, VecSpecialized {
13   override def length(): Int
14   override def apply(i: Int): Int
15   override def scalarProduct(other: VecSpecialized$$$sp$scala$Int): Int
16 }
17
18 class VecSpecialized$impl$scala$Int(
19   VecSpecialized$impl$scala$Int$$elems: Int[],
20   VecSpecialized$impl$scala$Int$$evidence$2: Numeric$$$sp$scala$Int) extends
21   Object(), VecSpecialized$$$sp$scala$Int(),
22   VecSpecialized(this.VecSpecialized$impl$scala$Int$$elems,
23     this.VecSpecialized$impl$scala$Int$$evidence$2),
24   VecSpecialized$$$sp$scala$Int {
25   private val VecSpecialized$$$elems: Int[] =
26     this.VecSpecialized$impl$scala$Int$$elems
27   private given val VecSpecialized$$$evidence$2: Numeric$$$sp$scala$Int =
28     this.VecSpecialized$impl$scala$Int$$evidence$2

```

```

29 private val VecSpecialized$$num: Numeric$$sp$scala$Int =
30   this.VecSpecialized$$evidence$2
31 override def length(): Int = this.VecSpecialized$$elems.[]length()
32 override def apply(i: Int): Int = this.VecSpecialized$$elems.[]apply(i)
33 override def scalarProduct(other: VecSpecialized$$sp$scala$Int): Int =
34   {
35     require(this.length() == other.length())
36     var result: Int = this.VecSpecialized$$num.fromInt(0)
37     scala.runtime.RichInt.until$extension(intWrapper(0), this.length()).
38       foreach((i: Int) =>
39         result =
40           this.VecSpecialized$$num.plus(result,
41             this.VecSpecialized$$num.times(this.apply(i), other.apply(i)))
42       )
43     result: Int
44   }
45 private val VecSpecialized$impl$scala$Int$$elems: Int[]
46 private given val VecSpecialized$impl$scala$Int$$evidence$2:
47   Numeric$$sp$scala$Int
48 override def apply(i: Int): Object = Int.box(this.apply(i))
49 override def scalarProduct(other: VecSpecialized): Object =
50   Int.box(
51     this.scalarProduct(other.asInstanceOf[VecSpecialized$$sp$scala$Int]))
52 }

```

Listing 46: Specialization through specialized traits after erasure

As we only generate specializations for the type combinations used in the user’s code rather than pre-generating them we have vastly fewer specializations than Scala 2’s `@specialized` annotation. This avoids the exponential blowup and means the solution would in principle be suitable for application in a number of cases where this was infeasible previously, such as tuples of size greater than 2.

6.3 Ergonomics

There are a few limitations of the specialized traits feature which make it less practical to write specialized code than would be desired. These are discussed here and could be addressed in future iterations of the feature.

- **Pattern Matching:** Pattern matching on specialized traits requires us to use the *has-type* style. We do not have case classes. This makes pattern matching less ergonomic than desired, and generates annoyances related to unchecked pattern matching of argument types, as shown in Listing 47. It is difficult to avoid the *has-type* pattern matching via a `Boolean`-returning `unapply` method without pattern-match exhaustivity warnings.

```

1 sealed inline trait List[+T: Specialized]:
2   def foreach[S](f: T => Unit): Unit = (this: List[T]) match {
3     case xs: ::[T @unchecked] => f(xs.head); xs.tail.foreach(f)
4     case _: Nil[_] =>
5   }

```

Listing 47: Pattern matching of specialized traits

- **Requirement for Specialized Methods to be Inline:** This makes writing recursive methods more tricky, although they can be written by defining a nested function, as in Listing 48, but this becomes a problem when the type needs to change in the recursion, as in Listing 49.

```

1 sealed inline trait List[+T: Specialized]:
2   inline def zip[S: Specialized](other: List[S]): List[(T, S)] =
3     def zip(xxs: List[T], yys: List[S]): List[(T, S)] = (xxs, yys) match {
4       case (_, Nil[_], _) => Nil()
5       case (_, _, Nil[_]) => Nil()
6       case (xxs: List[T @unchecked], yys: List[S @unchecked]) => (xxs.head, ←
          yys.head), zip(xxs.tail, yys.tail))
7     }
8   zip(this, other)

```

Listing 48: Recursive specialized methods

```

1 inline def eval[A: Specialized](node: Node[A]) = node match {
2   case n: Source[_]      => n.data
3   case n: MapNode[_]     => eval(n.parent).map(n.f)
4   case n: FilterNode[_]  => eval(n.parent).filter(n.f)
5 }

```

Listing 49: Type must change in recursion; this is unimplementable

Evaluating a `MapNode` for a function ($f: B \rightarrow A$) requires us to evaluate a node of type `B` to then evaluate the `MapNode[A, B]: Node[A]`. This is impossible with a monomorphic nested function like in Listing 48, but if implemented as in Listing 49 the inlining will continue forever.

7 Future Work

In this section we detail the remaining work on the specialized traits feature to make it releasable, and elaborate on some potential improvements to its utility, ergonomics and organisation. Along with the future work on inline traits in section 4.7, these form the next steps for bringing specialization to Scala 3.

7.1 Suggested Fixes & Checks

- **Fixes to Partial Specializations:** Currently partial specialization sometimes generates an additional specialization to `Any` which is not correct, and has trouble with methods in the body of a partially specialized trait. These bugs need to be ironed out before release. By changing the condition for specialization from *at least one argument is a concrete type* to *all specialized arguments are concrete types* this issue should be fixed.
- **Reduce Class Name Length:** Because the `$sp$` traits and `$impl$` classes are named using the fully qualified names (e.g. `scala.Int`) of the types used to specialize them their names can get quite long., but this risks running into the JVM maximum class name length (255 chars), and these names can also be challenging to read. It may be advisable to adopt the naming scheme from Scala 2 specialization for primitives, in which e.g. `Int` is written as `I`. Reference type names could be hashed if necessary.
- **Better Miniphase Processing:** To avoid unnecessary passes over the abstract syntax tree being compiled, the Scala 3 compiler supports miniphases, introduced in [7], which fuse a number of tree transformations into a single traversal.
DesugarSpecializedTraits was written as a miniphase, and *specializeInlineTraits* converted to be a miniphase, however the current implementations of these phases do not make the most of miniphase processing as they still fold/traverse the entire AST in their miniphase processing. This could be improved by moving some operations out into `transform_` methods.
- **Correct Integration with Build Tools:** We need to check that the generated classes are invalidated correctly so that tools like SBT know when they need to be regenerated to avoid stale classes being reused and forcing a clean build on the user. Furthermore, if library `A` and library `B` both specialize `MathLib`, and user code uses both libraries `A` and `B` do we have a mechanism for resolving which specialization to use?
- **Better Error Message Positioning:** If an error arises in code inlined into an `$sp$` trait or `$impl$` class, we typically report the error at the location that caused the specialization, e.g. the variable of type `Vec[Int]`. We don't currently have a mechanism to also show the inlining stack in these cases, which can make error message positioning quite cryptic. However, this is restricted to errors that occur after *desugarSpecializedTraits* and not in the original inline trait code, which is relatively uncommon.

7.2 Desirable Future Features

Many of these were initially described in [6]:

- **Caching of Specialized Traits:** The current implementation generates only one instance of each `$sp$` trait and `$impl$` class no matter how many times the corresponding specialization is referenced in the program. However, if the same specialization is mentioned in multiple compilation units, it will be regenerated (and the old version overwritten). It should be possible to avoid the extra work of regenerating the specialized version.
- **Greater Specialization to Reference Types:** As discussed in the explanation of the `PairHashing` benchmark, the `SpecType` mechanism currently limits the benefits we can get from avoiding megamorphic dispatch. Perhaps it is more desirable to abandon supporting variance for specialized traits, and allow fully precise specialization to any reference type (without type parameters).
- **Adapt Overloading to Specialization:** In this way, we could freely convert between collections of specialized and non-specialized type. This would entail supporting unspecialized methods with the same signature as specialized methods, and dispatching to the correct one. Listing 50 shows how this would allow us to map over a specialized collection even if we don't statically know the return type of the mapped function.

```
1 inline trait faster.Vector[A: Specialized]:  
2   inline def map[B: Specialized] =  
3     new faster.Vector[B](elems.map(f))  
4  
5   def map[B](f: A => B): ordinary.Vector[B] =  
6     new ordinary.Vector[B](elems.map(f))
```

Listing 50: Adapting overloading to specialization

This can be extended to generate these methods automatically and dispatch at runtime via path splitting to get a maximum of ergonomics while maintaining good performance. The details are discussed in [6].

- **Support Overloading across Specializations:** The code in Listing 51 is rejected because historically both overloads of `method1` would erase to the same type. With specialization we could permit this, and resolve a typical gripe that overloading is not allowed on type arguments in Scala.

```
1 inline trait Foo[T: Specialized]  
2 def method1(x: Foo[Int]) = 10  
3 def method1(x: Foo[String]) = 10
```

Listing 51: Overloading between different specializations

- **Specializing Tuples:** The Scala 2 scheme was not conducive to specializing tuples of arity beyond 2 due to the exponential blowup in generated classes. This scheme, on the other hand, would be suitable, and may lead to a performance gain.
- **Hand-written Specializations:** Allow the user to manually define the implementation for e.g. `Foo$impl$Int` by allowing them to optimise a specialized subtrait (this is already supported), and automatically replacing instances of `Foo$sp$Int` / `Foo$impl$Int` with that specialized subtrait. For example, replace references to `HashMap$sp$Int$String` with `IntHashMap$sp$String` under Listing 52:

```

1 inline trait HashMap[K: Specialized, +V: Specialized]
2 inline trait IntHashMap[+V: Specialized] extends HashMap[Int, V]:
3   // faster algorithm under K = Int

```

Listing 52: Hand-written specializations

- **Specialized Standard Library:** It should be possible to specialize a number of the container classes in the standard library, and thus gain efficiency improvements for any user of these containers. There is some precedent for this in other languages such as Kotlin, as discussed in [6].

A Benchmark Code

In each case we provide the benchmark code for the monomorphic case. The code for the polymorphic case is similar and can be found in the PR.

A.1 VectorDotProduct

```
1  //> using scala 3.10.0-RC1-bin-SNAPSHOT
2  //> using options -language:experimental.specializedTraits
3
4  package dotty.tools.benchmarks.specializedtraits
5
6  import org.openjdk.jmh.annotations.*
7  import java.util.concurrent.TimeUnit
8
9  class VecManual(elems: Array[Int]):
10     private val num = summon[Numeric[Int]]
11
12     def length = elems.length
13
14     def apply(i: Int): Int = elems(i)
15
16     def scalarProduct(other: VecManual): Int =
17         require(this.length == other.length)
18         var result = num.fromInt(0)
19         for i ← 0 until length do
20             result = num.plus(result, num.times(this(i), other(i)))
21         result
22
23  class VecGeneric[T: Numeric](elems: Array[T]):
24     private val num = summon[Numeric[T]]
25
26     def length = elems.length
27
28     def apply(i: Int): T = elems(i)
29
30     def scalarProduct(other: VecGeneric[T]): T =
31         require(this.length == other.length)
32         var result = num.fromInt(0)
33         for i ← 0 until length do
34             result = num.plus(result, num.times(this(i), other(i)))
35         result
36
37  inline trait VecSpecialized[T: {Specialized, Numeric2}](elems: Array[T]):
38     private val num = summon[Numeric2[T]]
39
40     def length = elems.length
41
42     def apply(i: Int): T = elems(i)
43
44     def scalarProduct(other: VecSpecialized[T]): T =
45         require(this.length == other.length)
46         var result = num.fromInt(0)
47         for i ← 0 until length do
```

```

48         result = num.plus(result, num.times(this(i), other(i)))
49         result
50
51     @State(Scope.Benchmark)
52     class Arrays:
53         val arr1 = Array.fill(100_000_000) {math.round(math.random().floatValue * 4)}
54         val arr2 = Array.fill(100_000_000) {math.round(math.random().floatValue * 4)}
55         val target = arr1.zip(arr2).map((x, y) => x * y).fold(0)(_ + _)
56
57     @State(Scope.Benchmark)
58     @BenchmarkMode(Array(Mode.AverageTime))
59     @OutputTimeUnit(TimeUnit.MILLISECONDS)
60     @Warmup(iterations = 5, time = 10, timeUnit = TimeUnit.SECONDS)
61     @Measurement(iterations = 15, time = 10, timeUnit = TimeUnit.SECONDS)
62     @Fork(1)
63     class VecBench:
64         @Benchmark
65         def manual(arr: Arrays) =
66             val x = VecManual(arr.arr1)
67             val y = VecManual(arr.arr2)
68             assert(x.scalarProduct(y) == arr.target)
69
70         @Benchmark
71         def generic(arr: Arrays) =
72             val x = VecGeneric[Int](arr.arr1)
73             val y = VecGeneric[Int](arr.arr2)
74             assert(x.scalarProduct(y) == arr.target)
75
76         @Benchmark
77         def specialized(arr: Arrays) =
78             val x = new VecSpecialized[Int](arr.arr1) {}
79             val y = new VecSpecialized[Int](arr.arr2) {}
80             assert(x.scalarProduct(y) == arr.target)
81
82     inline trait Numeric2[T: Specialized]:
83         def fromInt(x: Int): T
84         def plus(x: T, y: T): T
85         def times(x: T, y: T): T
86
87     implicit object IntIsNumeric extends Numeric2[Int]:
88         override def fromInt(x: Int): Int = x
89         override def plus(x: Int, y: Int): Int = x + y
90         override def times(x: Int, y: Int): Int = x * y

```

A.2 MatrixMultiplication

```
1  >>> using scala 3.10.0-RC1-bin-SNAPSHOT
2  >>> using options -language:experimental.specializedTraits
3
4  package dotty.tools.benchmarks.specializedtraits
5
6  import org.openjdk.jmh.annotations.*
7  import java.util.concurrent.TimeUnit
8  import org.openjdk.jmh.infra.Blackhole
9
10 // Flat row-major layout: element (i,j) of an n x n matrix is at index i*n+j
11 class MatManual(val elems: Array[Int], val n: Int):
12   def apply(i: Int, j: Int): Int = elems(i * n + j)
13
14   def matMul(other: MatManual, result: Array[Int]): MatManual =
15     require(this.n == other.n)
16     for
17       i ← 0 until n
18       k ← 0 until n
19     do
20       val aik = this(i, k)
21       for j ← 0 until n do
22         result(i * n + j) += aik * other(k, j)
23     MatManual(result, n)
24
25 class MatGeneric[T: Numeric](val elems: Array[T], val n: Int):
26   private val num = summon[Numeric[T]]
27
28   def apply(i: Int, j: Int): T = elems(i * n + j)
29
30   def matMul(other: MatGeneric[T], result: Array[T]): MatGeneric[T] =
31     require(this.n == other.n)
32     for
33       i ← 0 until n
34       k ← 0 until n
35     do
36       val aik = this(i, k)
37       for j ← 0 until n do
38         result(i * n + j) = num.plus(result(i * n + j), num.times(aik, other(k, j)))
39     MatGeneric(result, n)
40
41 inline trait MatSpecialized[T: {Specialized, NumericSpecialized}](val elems: Array[T], ↵
42   val n: Int):
43   private val num = summon[NumericSpecialized[T]]
44
45   def apply(i: Int, j: Int): T = elems(i * n + j)
46
47   def matMul(other: MatSpecialized[T], result: Array[T]): Array[T] =
48     require(this.n == other.n)
49     for
50       i ← 0 until n
51       k ← 0 until n
52     do
53       val aik = this(i, k)
54       for j ← 0 until n do
```

```

54         result(i * n + j) = num.plus(result(i * n + j), num.times(aik, other(k, j)))
55     result
56
57 @State(Scope.Benchmark)
58 class Matrices:
59     val n = 300
60
61     val mat1 = Array.fill(n * n)(math.round(math.random() * 100).toInt)
62     val mat2 = Array.fill(n * n)(math.round(math.random() * 100).toInt)
63
64     val result = Array.ofDim[Int](n * n)
65
66 @State(Scope.Benchmark)
67 @BenchmarkMode(Array(Mode.AverageTime))
68 @OutputTimeUnit(TimeUnit.MILLISECONDS)
69 @Warmup(iterations = 5, time = 10, timeUnit = TimeUnit.SECONDS)
70 @Measurement(iterations = 15, time = 10, timeUnit = TimeUnit.SECONDS)
71 @Fork(1)
72 class MatBench:
73     @Benchmark
74     def manual(m: Matrices, blackHole: Blackhole) =
75         val x1 = MatManual(m.mat1, m.n)
76         val y1 = MatManual(m.mat2, m.n)
77         blackHole.consume(x1.matMul(y1, m.result))
78
79     @Benchmark
80     def generic(m: Matrices, blackHole: Blackhole) =
81         val x1 = MatGeneric[Int](m.mat1, m.n)
82         val y1 = MatGeneric[Int](m.mat2, m.n)
83         blackHole.consume(x1.matMul(y1, m.result))
84
85     @Benchmark
86     def specialized(m: Matrices, blackHole: Blackhole) =
87         val x1 = new MatSpecialized[Int](m.mat1, m.n) {}
88         val y1 = new MatSpecialized[Int](m.mat2, m.n) {}
89         blackHole.consume(x1.matMul(y1, m.result))
90
91 inline trait NumericSpecialized[T: Specialized]:
92     def fromInt(x: Int): T
93     def plus(x: T, y: T): T
94     def times(x: T, y: T): T
95
96 implicit object IntIsNumeric extends NumericSpecialized[Int]:
97     override def fromInt(x: Int): Int = x
98     override def plus(x: Int, y: Int): Int = x + y
99     override def times(x: Int, y: Int): Int = x * y

```

A.3 PairHashing

```
1  //> using scala 3.10.0-RC1-bin-SNAPSHOT
2  //> using options -language:experimental.specializedTraits
3
4  package dotted.tools.benchmarks.specializedtraits
5
6  import org.openjdk.jmh.annotations.*
7  import java.util.concurrent.TimeUnit
8  import scala.util.Random
9  import org.openjdk.jmh.infra.Blackhole
10
11 class PairGeneric[T, S](x: T, y: S):
12   override def hashCode(): Int = x.hashCode() ^ y.hashCode()
13
14 inline trait PairSpecialized[T: Specialized, S: Specialized](x: T, y: S):
15   override def hashCode(): Int = x.hashCode() ^ y.hashCode()
16
17 class PairStringString(x: String, y: String):
18   override def hashCode(): Int = x.hashCode() ^ y.hashCode()
19
20 @State(Scope.Benchmark)
21 class Pairs:
22   val n = 5_000_000
23   val Ms_str_str = (0 until n).map(i => PairStringString("Good Morning", i.toString()))<->
24     .toArray
25   val As_str_str = (0 until n).map(i => PairGeneric("Good Morning", i.toString()))<->
26     .toArray
27   val Ts_str_str = (0 until n).map(i => new PairSpecialized[String, String]("Good <->
28     Morning", i.toString()) {}).asInstanceOf[Any].toArray
29
30 @State(Scope.Benchmark)
31 @BenchmarkMode(Array(Mode.AverageTime))
32 @OutputTimeUnit(TimeUnit.MILLISECONDS)
33 @Warmup(iterations = 5, time = 10, timeUnit = TimeUnit.SECONDS)
34 @Measurement(iterations = 15, time = 10, timeUnit = TimeUnit.SECONDS)
35 @Fork(1)
36 class PairHashBench:
37   @Benchmark
38   def manual(state: Pairs, blackHole: Blackhole) =
39     var i = 0
40     var res = Random.nextInt(10)
41     while (i < state.n) {
42       res ^= state.Ms_str_str(i).hashCode()
43       i += 1
44     }
45     blackHole.consume(res)
46
47   @Benchmark
48   def generic(state: Pairs, blackHole: Blackhole) =
49     var i = 0
50     var res = Random.nextInt(10)
51     while (i < state.n) {
52       res ^= state.As_str_str(i).hashCode()
53       i += 1
54     }
55     }
```

```

52     blackHole.consume(res)
53
54     @Benchmark
55     def specialized(state: Pairs, blackHole: Blackhole) =
56         var i = 0
57         var res = Random.nextInt(10)
58         while (i < state.n) {
59             res ^= state.Ts_str_str(i).asInstanceOf[PairSpecialized[String, String]].hashCode←→
60                 ()
61             i += 1
62         }
63         blackHole.consume(res)

```

Note: Because of the `ClassTag` issues mentioned in section 6 we type the `Array` as `Array[Any]` and cast when extracting values. This doesn't affect the benchmark.

References

- [1] Timothée Andres. “Inline Traits for Specialization in Scala 3”. In: *EPFL Master Thesis* (2023). URL: <https://infoscience.epfl.ch/handle/20.500.14299/199047>.
- [2] Matt D’Souza et al. “Tastytruffle: Just-in-time specialization of parametric polymorphism”. In: *Proceedings of the ACM on Programming Languages* 7.OOPSLA2 (Oct. 2023), pp. 1561–1588. DOI: 10.1145/3622853.
- [3] Iulian Dragos and Martin Odersky. “Compiling generics through user-directed type specialization”. In: *Proceedings of the 4th workshop on the Implementation, Compilation, Optimization of Object-Oriented Languages and Programming Systems* (July 2009), pp. 42–47. DOI: 10.1145/1565824.1565830.
- [4] Martin Odersky. *Revive or replace @specialized?* June 2022. URL: <https://github.com/scala/scala3/issues/15532>.
- [5] Martin Odersky. *Specialization for scala 3*. June 2023. URL: <https://github.com/scala/scala3/discussions/18044>.
- [6] Martin Odersky. *Specialized traits design draft*. Nov. 2024. URL: <https://github.com/scala/scala3/pull/21950>.
- [7] Dmitry Petrashko, Ondřej Lhoták, and Martin Odersky. “Miniphases: Compilation using modular and efficient tree transformations”. In: *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation* (June 2017), pp. 201–216. DOI: 10.1145/3062341.3062346.

Artificial Intelligence Integrity Statement

The body text of this report is entirely written by the author. However, some AI tools including ChatGPT, Claude Code and Google Gemini were used during the project to:

- Brainstorm ideas.
- Assist with debugging of the project code.
- Generate some illustrative examples and benchmarks to exercise the new feature, including the `MatrixMultiplication` benchmark and the bridges example in Listing 43.
- Support with LaTeX code, including for the performance evaluation graphs.